

CLAUDE.md is unique because it isn't a traditional README written to inform human developers. It is a behavioral contract written directly for the AI, loaded as a system prompt at the start of every session.

If your file is too long or filled with general knowledge, the agent will suffer from context bloat and start ignoring your rules.

Here is the best way to structure it to keep the agent focused and efficient.

1. Keep it Short and Hardcoded (Under 150 Lines)

Frontier models only reliably follow about 150-200 distinct instructions simultaneously. Claude Code's own internal system prompt already takes up about 50 of those slots.

- **Be brutal with pruning:** Your file should ideally be between 80 and 120 lines. Every line beyond that competes with the lines that actually matter.
- **Don't teach it basic coding:** The agent already knows standard TypeScript or Python conventions. Only include rules that differ from defaults (e.g., "Use snake_case for API routes but camelCase for database models").

2. Structure by "Rule vs. Map"

Instead of dumping every single guideline into one massive file, structure your CLAUDE.md as a **map** that points to deeper context when needed.

Your root file should look something like this:

Markdown

1. Hard Rules (Always Enforced)

- Never run database migrations without creating a backup first
- Git Bash on Windows: always use forward slashes
- Before writing any code, state the verification method

2. Project Gotchas

- The authentication module does X unconventionally because of legacy reason Y
- (Use this section for recurring mistakes the agent keeps making)

3. Context Pointers (Read Only When Relevant)

- Working on the UI? Read '@docs/ui-conventions.md'
- Touching the database? Read '@docs/schema-rules.md'
- Git workflow instructions: '@docs/git-instructions.md'

By pointing to specific file paths (@docs/...), the agent will only load those specific conventions when it actually touches those areas of the codebase.

3. Include Specific Execution Commands

The agent cannot guess your exact build tools. Give it the exact CLI commands it needs to test and verify its own work.

Include exact strings:

- `npm run test:unit` (instead of just "run tests")
- `pnpm tsc --noEmit`
- `gh issue view` (for checking GitHub context)

4. Keep "Ephemeral Memory" Out

A common mistake is letting the agent automatically append "what we did today" or "how these modules connect" into the CLAUDE.md file.

This is a trap. CLAUDE.md is for **static, permanent rules**. If the agent wants to document what it learned about your codebase, tell it to write that to a separate `agent_docs/` directory or a cache file, and have the CLAUDE.md point to it.

Think of it like this: CLAUDE.md is the senior engineer's exact marching orders. The more focused the brief, the better the output.